

Lab 1: Asynchronous File Processing and CLI Scripting with Node.js

Course: Distributed Systems & Server-Side Engineering

Duration: 50 Minutes

Focus: Non-blocking I/O, file system operations (node:fs/promises), cross -platform path handling (node:path), and POSIX-compliant CLI argument parsing.

Objectives

- Manipulate filesystem hierarchies asynchronously using modern Promise -based APIs.
- Implement robust path normalization to avoid cross -platform runtime failures.
- Construct a CLI utility capable of parsing positional arguments and handling I/O exceptions gracefully.
- Contrast sequential and concurrent asynchronous operations within the Node.js event loop.

Lab Timeline

Segment	Allocation	Objective
Phase 1	00 – 10 min	Environment initialization and test fixture generation
Phase 2	10–30 min	Script implementation (log - analyzer.mjs)
Phase 3	30–40 min	Executing test suites and edge -case validation
Phase 4	40–50 min	Reflexive analysis and short -answer documentation

Phase 1: Environment Setup & Test Fixtures (10 Min)

Initialize a clean workspace configured for ECMAScript Modules (ESM).

```
mkdir node-lab-1 && cd node-lab-1
npm init -y
npm pkg set type="module"
mkdir -p data/logs data/corrupt
```

Create a data population script named seed.mjs to generate dummy server log files:

```
// seed.mjs
import { writeFile } from 'node:fs/promises';
import path from 'node:path';

const logsDir = path.resolve('data', 'logs');

const files = {
  'auth.log': 'INFO: User login\nERROR: Invalid credentials\nINFO: User logout\n',
  'api.log': 'INFO: GET /api/v1 200\nINFO: POST /api/v1 201\nERROR: 500 Internal Error\nERROR: ',
  'db.log': 'INFO: Connection established\nINFO: Query executed\n',
  'notes.txt': 'This file should be ignored by your script.'
};

for (const [filename, content] of Object.entries(files)) {
  await writeFile(path.join(logsDir, filename), content, 'utf-8');
}

console.log('Fixtures written to data/logs/');
```

Run the seeder:

```
node seed.mjs
```

Phase 2: CLI Implementation (20 Min)

Build a CLI tool named log-analyzer.mjs.

Functional Requirements

1. **CLI Signature:** `node log-analyzer.mjs <target-dir> <output-file>`
 - `<target-dir>`: Directory containing log files to process.
 - `<output-file>`: File path where summary metrics should be saved as JSON.
2. **Argument Validation:** Exit with code 1 and a standard error message if either argument is missing.
3. **Path Safety:** Resolve all paths using `path.resolve` and `path.join` to prevent directory

traversal ambiguity and cross-platform path delimiter issues.

4. **Targeted Scanning:** Scan the directory asynchronously using `readdir`. Process **only** files with the `.log` extension (`path.extname`).
5. **Concurrency:** Process all matching files concurrently using `Promise.all` or `Promise.allSettled`.
6. **Data Extraction:** For each valid file, count total lines and lines containing "ERROR".
7. **Atomic Write:** Output a JSON report containing total files scanned, aggregate lines, aggregate error counts, and per -file stats.

Implementation Skeleton

Complete the missing logic marked with TODO:

```
// log-analyzer.mjs
import { readdir, readFile, writeFile } from 'node:fs/promises';
import path from 'node:path';
import process from 'node:process';

// 1. Argument Parsing & Validation
const [, targetDirArg, outputFileArg] = process.argv;

if (!targetDirArg || !outputFileArg) {
  console.error('Usage: node log-analyzer.mjs <target-dir> <output-file>');
  process.exit(1);
}

const targetDir = path.resolve(process.cwd(), targetDirArg);
const outputFile = path.resolve(process.cwd(), outputFileArg);

async function analyzeLogFile(filePath) {
  // TODO: Read the file, count lines and errors, return metrics object

  return {
    file: path.basename(filePath),
    totalLines: lines.length,
    errors: errorCount
  };
}
```

```

async function run() {
  try {
    // 2. Read directory entries
    const entries = await readdir(targetDir, { withFileTypes: true });

    // 3. Filter for .log files only
    const logFiles = entries
      .filter(entry => entry.isFile() && path.extname(entry.name) === '.log')
      .map(entry => path.join(targetDir, entry.name));

    // 4. Concurrently process files
    const fileStats = await Promise.all(logFiles.map(analyzeLogFile));

    // 5. Aggregate totals
    const report = {
      timestamp: new Date().toISOString(),
      directoryScanned: targetDir,
      filesProcessed: fileStats.length,
      aggregateLines: fileStats.reduce((acc, curr) => acc + curr.totalLines, 0),
      aggregateErrors: fileStats.reduce((acc, curr) => acc + curr.errors, 0),
      details: fileStats
    };

    // 6. Write output report
    await writeFile(outputFile, JSON.stringify(report, null, 2), 'utf-8');
    console.log(`Success: Report written to ${outputFile}`);

  } catch (error) {
    console.error(`Fatal I/O Error: ${error.message}`);
    process.exit(2);
  }
}

run();

```

Phase 3: Verification & Test Scenarios (10 Min)

Execute each test case via your terminal and record the behavior.

Test 1: Standard Execution (Happy Path)

Run the script against your seeded test directory:

```
node log-analyzer.mjs ./data/logs ./data/summary.json
```

- **Verify:** Check that ./data/summary.json was created. Confirm that notes.txt was ignored and that total files processed equals 3.

Test 2: Target Path Resolution Robustness

Execute the script from a subfolder or via relative parent traversals:

```
cd data && node ../log-analyzer.mjs logs/ ./summary-relative.json && cd ..
```

- **Verify:** Ensure path.resolve handles the dynamic process.cwd() without throwing a directory -not-found error.

Test 3: Missing Directory Fault Injection

Execute against a path that does not exist:

```
node log-analyzer.mjs ./non_existent_folder ./data/err.json
```

- **Verify:** The script catches the ENOENT system error code, outputs the formatted Fatal I/O Error: ... line to stderr, and exits with code 2 (confirm via echo \$?).

Test 4: Missing Arguments

Execute with omitted arguments:

```
node log-analyzer.mjs
```

- **Verify:** Correct usage string is logged and exit code is 1.

Phase 4: Reflexive Analysis Questions (10 Min)

Write your answers to the following questions in a file named REFLECTIONS.md.

1. **POSIX Resolution Mechanics (path.join vs. path.resolve):**
What is the specific architectural difference between path.join('/a', '../b') and path.resolve('/a', '../b')? Under what conditions would using path.join with user-supplied CLI arguments create an exploitable or fragile filesystem lookup?
2. **Concurrency and Backpressure:**
In Phase 2, Promise.all() was used to read all .log files concurrently. If data/logs/ contained 50,000 distinct log files, what specific OS -level and Node.js-level failure would occur (identify the specific POSIX error code)? How does replacing Promise.all with a worker pool, batching mechanism, or Node.js streams eliminate this failure mode?
3. **Event Loop Lifecycle:**
When readFile is called via node:fs/promises, which part of the Node.js runtime handles the actual disk read, and through which queue/mechanism does the resolved promise notify the V8 JavaScript thread?
4. **Failure Domains (Promise.all vs. Promise.allSettled):**
Suppose one file inside data/logs/ has read permissions revoked (chmod 000

data/logs/api.log). What happens to the output report using the current implementation?
Explain why an enterprise CLI tool should prefer `Promise.allSettled()` in this scenario.